

06-07-2025

Agenda - Database IV

- More SQL keywords / query.
- Run new SQL queries in pgAdmin4.
- working with cloud based postgres DB. → python

IN

let you check if a value matches any value in a list or subquery.

select column_name from table_name where artist_id IN ('12', '13', '14');

a = [1, 2, 3]

```
for i in range(0, 100, 1):  
    if i in a:  
        print(i)
```

→ 1
→ 2
→ 3

wifi-subscription:

call → provider → agent → select
→ DSID

IN (DS10, DS102)
↓

```
SELECT *  
FROM album  
WHERE artist_id IN (1,2,3,4,5)  
ORDER BY artist_id;
```

SQL command within Query / SubQuery

→ use a query inside another query,

② → select artist_id from artist where name
LIKE 'L%'

meaning → artist_id start from L

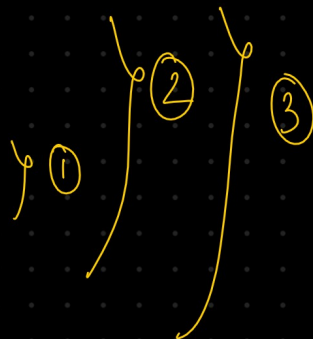
→ 10 artist id

① → select * from album where artist_id IN
(10 artist id)

→ select *
from album
where artist_id IN (SELECT artist_id from artist where
Name Like 'L%');

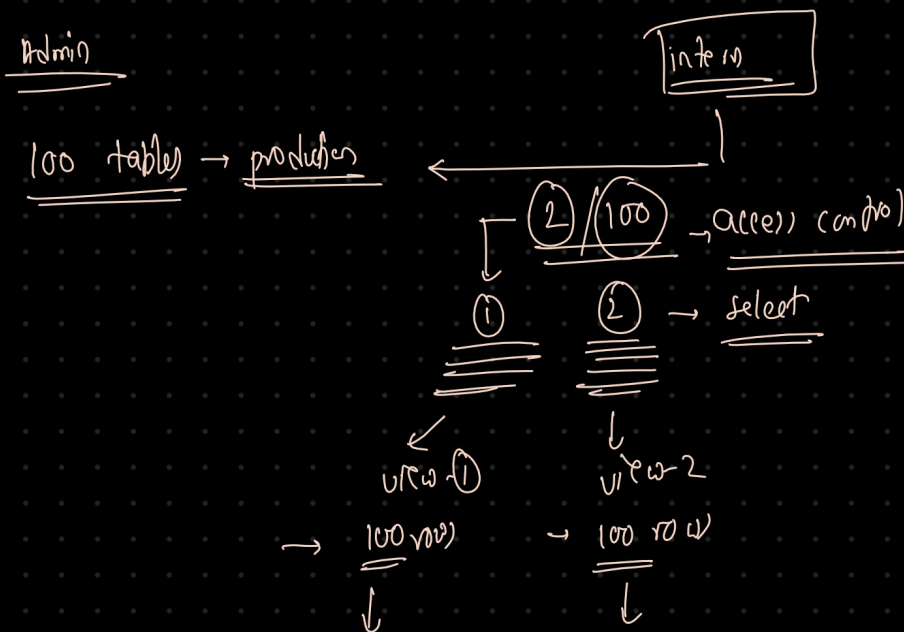
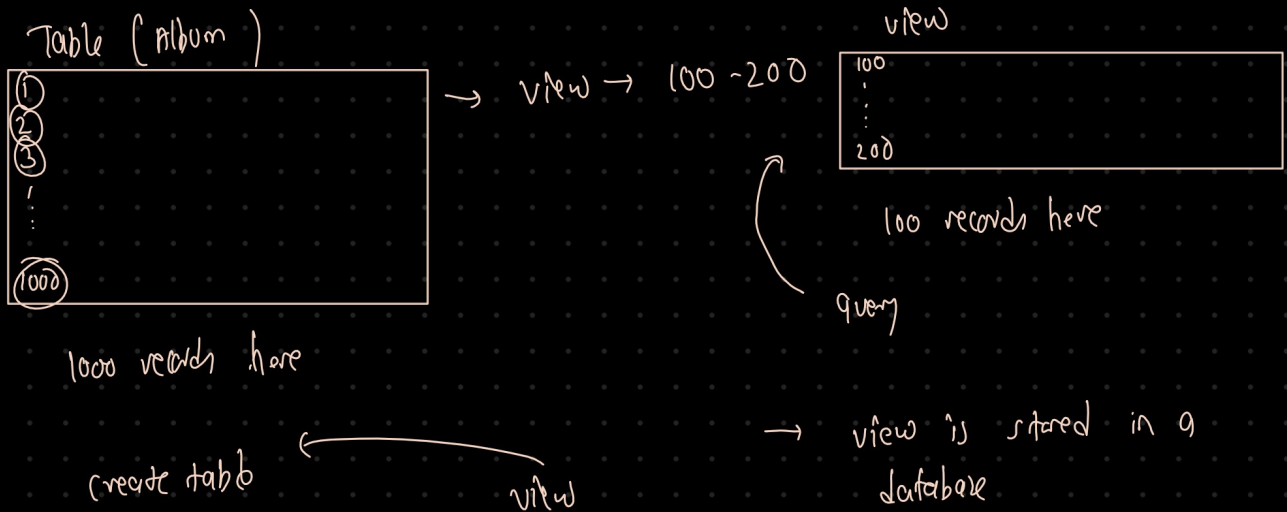
ILIKE → works on both case

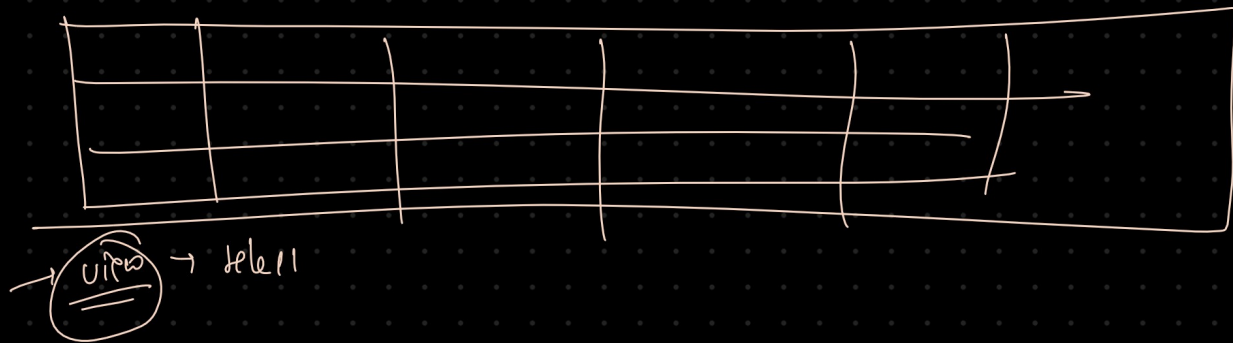
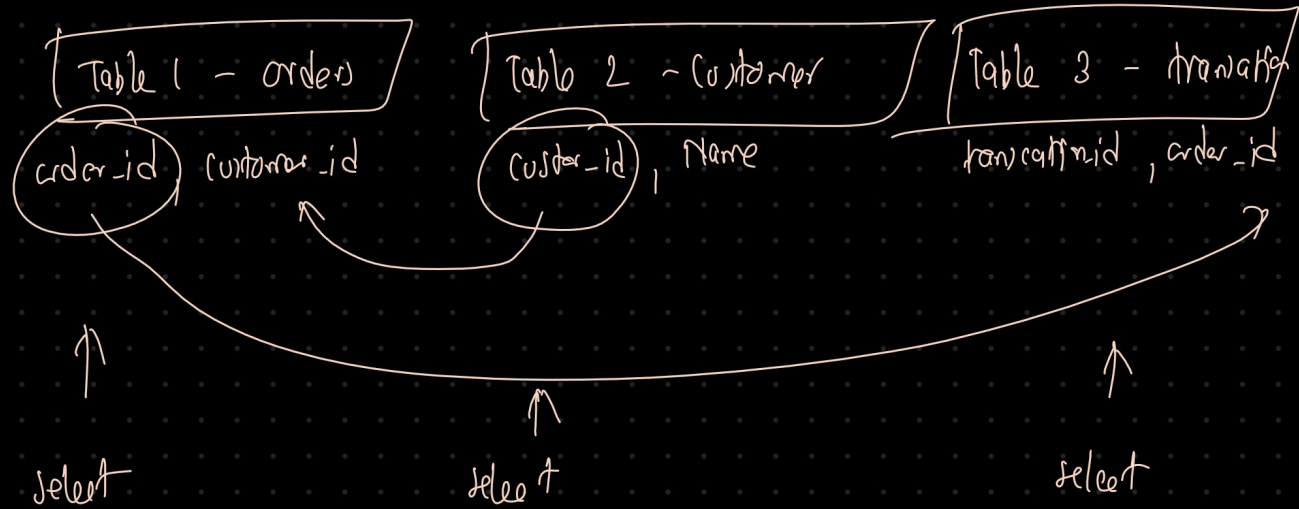
```
SELECT *  
FROM album  
WHERE artist_id IN (  
  SELECT artist_id  
  FROM artist  
  WHERE name ILIKE 'L%'  
)  
ORDER BY artist_id;
```



VIEW

- A view is a virtual table based on a query.
- It doesn't store data by itself, it shows live result whenever you query it.
- Once we have the view (not a table), we can query it like a table.





View

```
CREATE VIEW big_invoices AS
SELECT invoice_id, customer_id, total
FROM invoice
WHERE total > 20;
```

} create view

```
DROP VIEW big_invoices;
```

} Drop view

Advantage:

- simplify frequent joins and filters
- Improve readability
- Encapsulate business logic
- control access to data / data security

Definition of view:

```
select * from pg_views  
where viewname = 'big_invoices';
```

Advance

||

Concatenation operators

```
select 'Hello' || ' ' || 'World' AS greeting;
```

```
SELECT 'Hello' || ' ' || 'World' AS greeting;
```

name $\rightarrow$ artist

artist-id $\rightarrow$ artist

```
select name, artist-id from artist;
```

```
select artist.name, artist.artist-id from artist;
```

table-name.column-name (Syntax)

Joins:

Join combines rows from two or more table based on related column. (usually a foreign key)

- INNER JOIN

- LEFT JOIN (LEFT OUTER JOIN)

- RIGHT JOIN (RIGHT OUTER JOIN)

- FULL JOIN (FULL OUTER JOIN)

- CROSS JOIN

INNER JOIN

Returns rows where there is a match in both tables.

album →

①	②	③
—	—	—
—	—	—

artist → 1000 unique artist

① → ③
② → ②
③ → ①

album-id, title, name

p.s → list all albums with their artist name.

album			artist	
album-id	title	artist-id	artist-id	name
1	A	1	1	Mona
2	B	1	2	Aravind
3	C	3	3	Ann
4	D	4	4	Rahul
5	E	5	5	Lenovo
6	F	8	6	Bhuvileg
7	S	9	7	Shubham
		10	8	Rangga
			9	ben

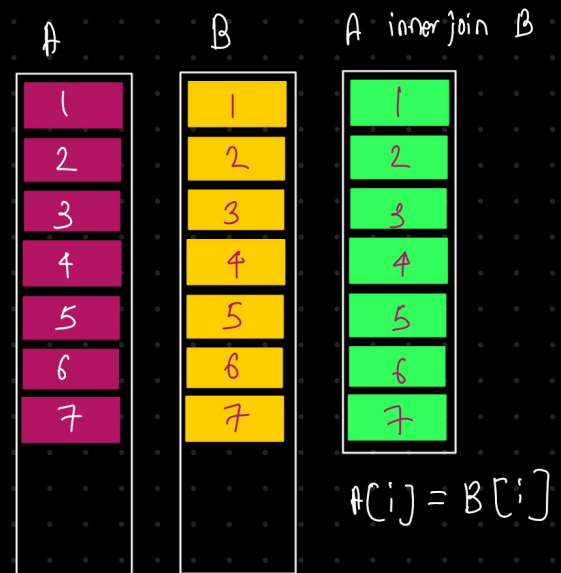
p.s → list all albums with their artist name.

select album.album-id, album.title, artist.name

get data only when both tables have that value

```
SELECT album.album_id, album.title, artist.name AS artist_name
FROM album
INNER JOIN artist
ON album.artist_id = artist.artist_id;
```

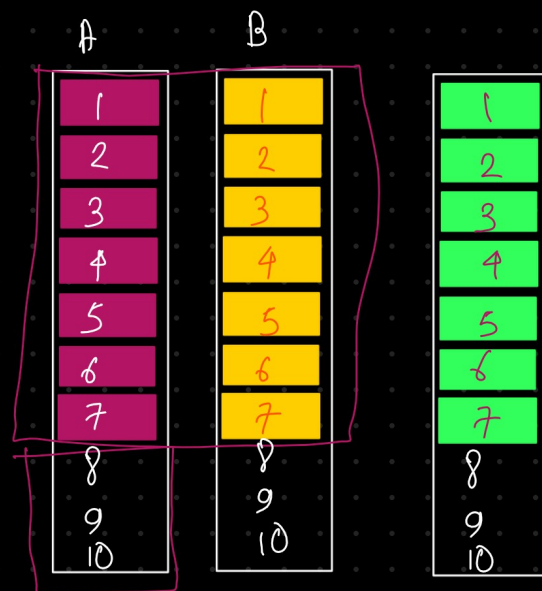
```
select al.album_id, al.title, art.name
from album al, artist art where
al.artist_id = art.artist_id;
```



LEFT JOIN

returns all rows from left table, plus matching rows from the right table.

If no match, the right side columns are Null



↓

	album-id	title	artist-id
1	1	A	1
2	2	B	2
3	3	C	3
4	4	D	2
5	5	E	10
6	6	F	10
7	7	G	10
8	8	H	10

1
2
3
4
5
6
7
8

↓

	artist-id	name
1	1	Mona1
2	2	Aravind
3	3	Arin
4	4	Rahul
5	5	Lenovo
6	6	Bhuvileg
7	7	Shubham
8	8	Kangga
9	9	ben

1
2
3
4
5
6
7
8
9

Inner Join → A inner join B on A.artist-id = B.artist-id

→	1	A	Mona1	→	4	D	Aravind	5 rows
→	2	B	Mona1	→	7	G	ben	
→	3	C	Aravind	→				

LEFT JOIN

A left join B on A.artist-id = B.artist-id

→	1	A	Mona1	→	4	D	Aravind	5
→	2	B	Mona1	→	7	G	ben	+
→	3	C	Aravind	→	6	F	NULL	3
→	5	E	NULL	→	8	H	NULL	

RIGHT JOIN

B RIGHT JOIN A ON A.artist_id = B.artist_id

↓

	album-id	title	artist-id
1	1	A	1
2	2	B	1
3	3	C	1
4	4	D	2
5	5	E	3
6	6	F	3
7	7	G	3
8	8	H	4

↓

	artist-id	name
1	1	Mona
2	2	Aravind
3	3	Arin
4	4	Rahul
5	5	Lenovo
6	6	Bhuvila
7	7	Shubham
8	8	Kanga
9	9	Ben

select artist.artist_id
artist.name
album.title

from album

Right JOIN artist

on album.artist_id = artist.artist_id

→ 1, mona, A
1, mona, B
2, Aravind, C
→ 2, Aravind, B
3, Arin, NULL
4, Rahul, NULL
5, Lenovo, NULL
6, Bhuvila, NULL
7, Shubham, NULL
8, Kanga, NULL
9, Ben, NULL

FULL OUTER JOIN

- return all rows from both table, matching rows where possible

→ Left A on B → 5 + 3

→ Right A on B → 5 + 4

→ 5 + 3 + 4 →

artist (A)				album (B)	
id	album-id			id	name
1	2	—	1	1	A
2	3	—	2	2	B
3	4	—	3	3	C
4	5	—	4	4	D
5	6	—	5	5	E
6	10	—	6	6	F
7	11	—	7	7	G
8	13	—	8	8	H
9	14	—	9	9	T

A-1	→	B-2
A-2	—	B-3
A-3	—	B-4
A-4	—	B-5
A-5	—	B-6

A-6	—	NULL
A-7	→	NULL
A-8	→	NULL
A-9	—	NULL

A

(common)		B
NULL	—	B-7
NULL	—	B-8
NULL	—	B-9
NULL	—	B-1

$$5 + 4 + 4 = \underline{\underline{13}}$$

```
SELECT
  artist.artist_id,
  artist.name AS artist_name,
  album.title AS album_title
FROM
  album
RIGHT JOIN artist
  ON album.artist_id = artist.artist_id;

-- Show all artists and their albums,
-- including artists with no albums.
```



```
select  artist.artist_id, artist.name,
        album.title
from
  album
```